

Repetition Structures

while structure

Syntax: while condition:

```

[statement]
[statement]

```

True
while block
False

* သတိပြုရန် အားလုံးသိရန်

Ex: num = 0
while num < 5:
 print(num)
 num += 2
 print("Done")

short-hand
num = num + 2

while block

Output: 0
2
4
Done

#1 num < 5 → 0 < 5
 print(num) → 0
 num += 2 → num = 2

#2 num < 5 → 2 < 5
 print(num) → 2
 num += 2 → num = 4

#3 num < 5 → 4 < 5
 print(num) → 4
 num += 2 → num = 6

#4 num < 5 → 6 < 5 False
 out of the loop!

Q19. အောက်ပါ ကုဒ်ကို အသုံးပြု၍ user မှတ်ပုံတင်မှုကို စစ်ဆေးပါ။
user မှတ်ပုံတင်မှုကို စစ်ဆေးပါ။

Ver 1

```

num = int(input())
total = 0
while num != -999999:
    total += num
    num = int(input())
print(total)

```

Ver 2

```

total = 0
while True:
    num = int(input())
    if num == -999999:
        break
    else:
        total += num
print(total)

```

* မှတ်ပုံတင်မှုကို စစ်ဆေးပါ။

Input Validation

Syntax:

* 2. Itam
66/67/68/69/70

Output: 20
40
60

↳ 2) รับจากชั้น User ลงถึง มัลติเพล็กซ์ n ครั้ง

e.g. $N=2$ ← จำนวน loop

1000 127

loop for

↑
number
↑
start end step

→ range(0, N, 1)

 $\hookrightarrow [0, 1, 2, \dots, n-1]$

* n loop!

print(f"Salary # 1 is {salary}")

range function

* function ~~for~~ sequence ~~on start~~!!

range(start, end, step) ↖ 0 values end

↳ e.x. range(0, 5, 1) → [0, 1, 2, 3, 4]
range(5, 1, -2) → [5, 3]

range(start, end) # step=1

↳ e.x. range(0, 4) → [0, 1, 2, 3]

range(end) # start=0, step=1

↳ e.x. range(3) → [0, 1, 2]

* ~~Q~~ ~~66n~~ ~~is~~ list ~~for~~ loop!

Ex

x = 5

y = 2

total = 0

for i in range(x, x, y//2): range(2, 5, 1) → [2, 3, 4]

total += i 0 + 2 + 3 + 4 = 9

print(total) → 9

Continue Statement

↳ ~~66n~~ ~~is~~ continue ~~Q~~ ~~4~~ ~~loop~~ [2, 3, 4, 5] (2, 3, 4, 5 loop)

Ex

n = 0

while n < 5:

n = n + 1

if n % 2 == 0:

#1

n = 1

if F

print(1)

#2

n = 2

if T

Ex

total = 0

for i in [1, 2, 3, 4, 5]:

if i % 2 == 0:

continue

<pre> if n % 2 == 0: continue print(n) </pre> Output: 1 3 5	<pre> # 2 n = 2 if T continue # 3 # 4 continue </pre>	<pre> if i > 2 == 20: continue total += i + 1 + 3 + 5 print(total) → 9 </pre>
--	---	--

List: data type ที่เก็บรายการข้อมูล

Syntax: [item1, item2, item3, item4]

* item เป็น data type ใดก็ได้ (string, int, float, ...)

e.g. [10, 20, "ABC", True, False]

Indexing: เรียงใน list มีตำแหน่งอะไรบ้าง

* ตำแหน่ง (index) เริ่มต้นที่ 0 | Syntax: list[index]

Ex my_list = [10, 20, 30]

print(my_list) → [10, 20, 30]

print(my_list[0]) → 10

↑
บอกค่า index 0 ของ my_list (indexing)

print(my_list[2]) → 30

your_list = ["A", "B", "C", "D"]

print(your_list[1]) → B

⊛ negative index:

- index ของสุดท้าย = -1 บอกย้อนกลับ!

index 0 1 2 3

Indexing

my_list = [10, 20, 30, 40]

index: 0 1 2 3

-4 -3 -2 -1

print(my_list[-1]) → 40

print(my_list[4]) → 10

⊗ if you index n't exist, it's error! (index out of range)

print(my_list[20]) × error!

Update list

• Update one index (update an existing index)

Syntax: list[index] = new_value

Ex: my_list = [10, 20, 30, 40]

print(my_list) → [10, 20, 30, 40]

my_list[0] = 3

print(my_list) → [3, 20, 30, 40]

my_list[1] = my_list[-1] * 2

print(my_list) → [3, 80, 30, 40]

my_list[0] = my_list[1] * 2 = 80 * 2 = 160

print(my_list) → [160, 80, 30, 40]

len() function

→ how many items are in the list ⇒ return the number of items in the list

my = [10, 20, 30, 40, 50]

l = len(my) = 5

```

my = [1, 2, 3, 4, 5]
l = len(my) = 5
print(l) → 5

```

List Operators

- + for list addition $[1, 2] + [3, 4] \rightarrow [1, 2, 3, 4]$
- * for list multiplication note: $[1, 2] * 3 \rightarrow [1, 2, 1, 2, 1, 2]$
repeated 3 times

EX

```

my_list = [1, 2]
your_list = [10, 20]
new_list = my_list + your_list → [1, 2, 10, 20]
print(new_list * 2) → [1, 2, 10, 20, 1, 2, 10, 20]

```

prohibit

for loop list

```

my_list = [10, 20, 30]
for num in my_list:
    print(num)

```

Output: 10
20
30

Ver 2. for loop over index

```

my_list = [10, 20, 30]
indexes = range(3)
indexes = range(len(my_list))
for i in indexes:
    print(my_list[i])

```

range(3) → [0, 1, 2]
range over my_list

Output: 10
20
30

```

#1
i=0
print(my_list[i]) → 10
#2
i=1
print(my_list[i]) → 20
#3
i=2
print(my_list[i]) → 30

```

for loop over index

```

my_list = [10, 20, 30]
for i in range(len(my_list)):
    print(my_list[i])

```

range(3) → [0, 1, 2]

```
print(my_list[i])
```

* both index & value both!

e.g. $[1, 2, 3, 4, 5] \xrightarrow{2} [1, 4, 9, 16, 25]$

```
nums = [1, 2, 3, 4, 5] ← update nums
```

```
for i in range(len(nums)):
```

```
    nums[i] = nums[i] ** 2
```

or

```
nums[0] = nums[0] ** 2  
nums[1] = nums[1] ** 2  
nums[2] = nums[2] ** 2  
...
```

```
print(nums) → [1, 4, 9, 16, 25]
```

Nested Loops

• nested

Ex

```
for i in [1, 2, 3]
```

```
    for j in [5, 6]
```

```
        print(i, j)
```

#1

i = 1

#1.1

j = 5

print(1, 5)

#1.2

j = 6

print(1, 6)

#2

i = 2

#2.1

j = 5

print(2, 5)

#2.2

j = 6

print(2, 6)

#3

i = 3

#3.1

j = 5

print(3, 5)

#3.2

j = 6

print(3, 6)

output

1 5

1 6

2 5

2 6

3 5

3 6

Ex

total = 0

```
for i in range(3):
```

```
    for j in [2, 5, 7]:
```

i

0 1 2

2 0 2 1 2 2 2

```

for j in [2, 5, 7]:
    if (i+j) % 2 == 0:
        total += (i*j)
print(total) → 16

```

2	0, 2 = 2	1, 2 = 3	2, 2 = 4
5	0, 5 = 5	1, 5 = 6	2, 5 = 7
7	0, 7 = 7	1, 7 = 8	2, 7 = 9

total = 5 + 7 + 4 = 16